LEMUEL DURAN FEBUARY/12/2026

BSIT-2A | ADVANCED DBMS LESSON 3 – STORED PROCEDURE & TRIGGER

SQL SCRIPT:

```
MariaDB [company_dbLCD]> DELIMITER $$
MariaDB [company_dbLCD]> CREATE PROCEDURE add_new_employee(
    -> IN emp_name VARCHAR(100),
    -> IN department_id INT,
    -> IN initial_salary DECIMAL(10,2)
    -> )
    -> BEGIN
    -> DECLARE new_emp_id INT;
    -> INSERT INTO employees (employee_name, dept_id, hire_date) VALUES (emp_name, department_id, CURDATE());
    -> SET new_emp_id = LAST_INSERT_ID();
    -> INSERT INTO salaries (employee_id, salary) VALUES (new_emp_id, initial_salary);
    -> END $$
Query OK, 0 rows affected (0.004 sec)
```

CALL:

```
MariaDB [company_dbLCD]> DELIMITER ;
MariaDB [company_dbLCD]> CALL add_new_employee('Juan Dela Cruz', 1, 30000);
Query OK, 2 rows affected (0.004 sec)

MariaDB [company_dbLCD]> CALL add_new_employee('Maria Santos', 2, 28000);
Query OK, 2 rows affected (0.005 sec)

MariaDB [company_dbLCD]> CALL add_new_employee('Pedro Reyes', 1, 25000);
Query OK, 2 rows affected (0.005 sec)
```

UPDATED TABLES:

```
MariaDB [company_dbLCD]> SELECT * FROM employees;
+-------------+---------------+---------+------------+
| employee_id | employee_name | dept_id | hire_date  |
+-------------+---------------+---------+------------+
|           2 | Bob Johnson   |       2 | 2019-11-10 |
|           3 | Charlie Brown |       2 | 2021-03-22 |
|           4 | Daisy Lee     |       3 | 2018-05-05 |
|           5 | Edward Harres |       4 | 2021-03-22 |
|           6 | Lemuel Duran  |       2 | 2026-04-20 |
|           7 | Lemuel Duran  |       2 | 2026-04-20 |
|           8 | Lee Du        |       2 | 2026-04-20 |
|           9 | Nics Aba      |       3 | 2026-04-20 |
|          10 | Lemuel Duran  |       2 | 2026-04-20 |
|          11 | Juan Dela Cruz|       1 | 2026-02-12 |
|          12 | Maria Santos  |       2 | 2026-02-12 |
|          13 | Pedro Reyes   |       1 | 2026-02-12 |
+-------------+---------------+---------+------------+
12 rows in set (0.001 sec)
```

```
MariaDB [company_dbLCD]> SELECT * FROM salaries;
+-------------+-----------+
| employee_id | salary    |
+-------------+-----------+
|           2 |  75000.00 |
|           3 |  68000.00 |
|           4 |  60000.00 |
|           5 |  10000.00 |
|           9 | 100000.00 |
|          11 |  30000.00 |
|          12 |  28000.00 |
|          13 |  25000.00 |
+-------------+-----------+
```

GUIDE QUESTIONS – ANSWERS

1. **Why is LAST_INSERT_ID() important in the stored procedure?**
   LAST_INSERT_ID() is important because it retrieves the automatically generated employee_id from the INSERT statement. When we insert a new employee, MySQL automatically assigns an employee_id (due to AUTO_INCREMENT). We need this ID to create the corresponding salary record in the salaries table. Without LAST_INSERT_ID(), we wouldn't know which employee_id to use when inserting the salary.
2. **What will happen if the department ID does not exist?**
   The stored procedure will fail with a FOREIGN KEY constraint error. This is because the employees table has a foreign key constraint that references departments(dept_id). MySQL will not allow inserting an employee with a department_id that doesn't exist in the departments table.
3. **How does a stored procedure reduce errors compared to manual INSERT statements?**
   A stored procedure reduces errors because the process is automated using one command. The user does not need to write multiple INSERT statements manually. This helps prevent typing mistakes and missing data.
4. **How many tables are affected when the stored procedure is executed?**
   Two tables are affected the employee table - new employee record is inserted and the salaries table - new salary record is inserted for that employee.

BONUS:

```
MariaDB [company_dbLCD]> DELIMITER $$
MariaDB [company_dbLCD]> CREATE PROCEDURE update_salary(
    -> IN emp_id INT,
    -> IN new_salary DECIMAL(10,2)
    -> )
    -> BEGIN
    -> UPDATE salaries SET salary = new_salary WHERE employee_id = emp_id;
    -> END $$
Query OK, 0 rows affected (0.005 sec)

MariaDB [company_dbLCD]> DELIMITER ;
MariaDB [company_dbLCD]> CALL update_salary(3, 35000);
Query OK, 1 row affected (0.007 sec)

MariaDB [company_dbLCD]> SELECT * FROM salaries WHERE employee_id = 3;
+-------------+----------+
| employee_id | salary   |
+-------------+----------+
|           3 | 35000.00 |
+-------------+----------+
1 row in set (0.004 sec)
```

SQL SCRIPT:

Trigger 1:

```
MariaDB [company_dbLCD]> DELIMITER $$
MariaDB [company_dbLCD]> CREATE TRIGGER after_employee_insert
    -> AFTER INSERT ON employees
    -> FOR EACH ROW
    -> BEGIN
    -> INSERT INTO salaries (employee_id, salary) VALUES (NEW.employee_id, 20000);
    -> END $$
Query OK, 0 rows affected (0.012 sec)

MariaDB [company_dbLCD]> DELIMITER ;
```

Trigger 2:

```
MariaDB [company_dbLCD]> DELIMITER $$
MariaDB [company_dbLCD]> CREATE TRIGGER before_salary_update
     -> BEFORE UPDATE ON salaries
     -> FOR EACH ROW
     -> BEGIN
     -> IF NEW.salary < 15000 THEN
     -> SET NEW.salary = OLD.salary;
     -> END IF;
     -> END $$
Query OK, 0 rows affected (0.007 sec)

MariaDB [company_dbLCD]> DELIMITER ;
```

Trigger 3:

```
MariaDB [company_dbLCD]> DELIMITER $$
MariaDB [company_dbLCD]> CREATE TRIGGER after_department_update
    -> AFTER UPDATE ON employees
    -> FOR EACH ROW
    -> BEGIN
    -> IF OLD.dept_id <> NEW.dept_id THEN
    -> UPDATE salaries SET salary = salary + 1000 WHERE employee_id = NEW.employee_id;
    -> END IF;
    -> END $$
Query OK, 0 rows affected (0.007 sec)

MariaDB [company_dbLCD]> DELIMITER ;
```

RESULT:

Trigger 1:

```
MariaDB [company_dbLCD]> INSERT INTO employees (employee_name, dept_id, hire_date) VALUES ('Nora Garcia', 3, CURDATE());

Query OK, 1 row affected (0.005 sec)

MariaDB [company_dbLCD]> SELECT * FROM SALARIES;
+-------------+-----------+
| employee_id | salary    |
+-------------+-----------+
|           2 |  76000.00 |
|           3 |  35000.00 |
|           4 |  60000.00 |
|           5 |  10000.00 |
|           9 | 100000.00 |
|          11 |  30000.00 |
|          12 |  28000.00 |
|          13 |  25000.00 |
|          14 |  20000.00 |
|          15 |  20000.00 |
|          16 |  20000.00 |
+-------------+-----------+
11 rows in set (0.000 sec)
```

Trigger 2:

```
MariaDB [company_dbLCD]>  UPDATE salaries SET salary = 12000 WHERE employee_id = 2;
Query OK, 0 rows affected (0.002 sec)
Rows matched: 1  Changed: 0  Warnings: 0

MariaDB [company_dbLCD]> SELECT * FROM SALARIES;
+-------------+-----------+
| employee_id | salary    |
+-------------+-----------+
|           2 |  76000.00 |
|           3 |  35000.00 |
|           4 |  60000.00 |
|           5 |  10000.00 |
|           9 | 100000.00 |
|          11 |  30000.00 |
|          12 |  28000.00 |
|          13 |  25000.00 |
|          14 |  20000.00 |
|          15 |  20000.00 |
|          16 |  20000.00 |
+-------------+-----------+
11 rows in set (0.079 sec)
```

Trigger 3:

```
MariaDB [company_dbLCD]> UPDATE employees SET dept_id = 1 WHERE employee_id = 2;
Query OK, 1 row affected (0.003 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [company_dbLCD]> SELECT * FROM SALARIES;
+-------------+-----------+
| employee_id | salary    |
+-------------+-----------+
|           2 |  77000.00 |
|           3 |  35000.00 |
|           4 |  60000.00 |
|           5 |  10000.00 |
|           9 | 100000.00 |
|          11 |  30000.00 |
|          12 |  28000.00 |
|          13 |  25000.00 |
|          14 |  20000.00 |
|          15 |  20000.00 |
|          16 |  20000.00 |
+-------------+-----------+
11 rows in set (0.001 sec)
```

GUIDE QUESTIONS:

1. **Why can't triggers be executed using CALL?**
   Triggers cannot be executed using CALL because they are not stored procedures. Triggers are automatically executed by specific database events (INSERT, UPDATE, DELETE). They fire automatically when the triggering event occurs.
2. **What is the difference between OLD.department_id and NEW.department_id?**
   OLD.department_id is the value before the update happens. NEW.department_id is the value after the update. They are used to check if the department was changed.
3. **Why is BEFORE UPDATE used to prevent salary reduction?**
   BEFORE UPDATE is used because it allows us to modify the NEW values BEFORE they are actually written to the database. This lets us intercept and change the update. If we used AFTER UPDATE, the low salary would already be saved to the database, and we would need another UPDATE statement to fix it.
4. **What happens if multiple rows are updated at once?**

If multiple rows are updated at once, the trigger fires FOR EACH ROW individually. This means the trigger executes once for EACH row that is affected by the UPDATE statement.